Python Full Handbook

PART 2

1. Functions in Python

★ What is a Function ?

A function is a block of code that performs a specific task.
It helps us to write reusable, organized and easy to maintain code.

★ Why use Functions ?

- Reusability : Write once, use many times.
- Better Organization of code.
- Improves Readability.
- Easy to Test and Debug.

Built-in vs User-defined Functions

Built-in Functions	User-defined Functions
Predefined functions in Python that are always available.	Functions created by the user as per their requirement.
Example : print(), len(), type() range(), input() etc.	Example : def add(); def greet(): etc.

★ Syntax

```
def function_name(parameters):
    # block of code (function body)
    return value
```

Note :
def keyword is used to define a function.
return keyword is used to send a result back.

★ Function Calling

Once a function is defined, we can call (code) it whenever needed.

Example :
```
def greet():
    print("Hello, Python!")
greet()  # function call
```

★ Parameters vs Arguments

Parameters	Arguments
Variables listed in the function definition. (Placeholders)	Actual values passed to the function when it is called. (Real values)

Example :
```
def add(a, b):      # a, b → parameters
    return a + b
result = add(5, 3)  # 5, 3 → arguments
print(result)       # Output : 8
```

★ Default Arguments

We can provide a default value to a parameter
If no value is passed, the default value is used.

Example :
```
def greet(name = "Guest"):
    print("Hello,", name)
greet()          # Output : Hello, Guest
greet("Alice")   # Output : Hello, Alice
```

★ Return Statement

The return statement is used to send a result back from the function.

Example :
```
def add(a  b):
    return a + b
sum = add(10, 20)
print(sum)    # Output : 30
```

★ Simple Example

```
def square(x):
    return x * x
num = 6
ans = square(num)
print(ans)      # Output : 36
```

→ Define function
→ Pass value
→ Get result

Key Point :
- ✓ Functions reduce repetition.
- ✓ Functions make code modular.
- ✓ Always use meaningful function names.

Python Full Handbook

PART 2

2. STRINGS IN PYTHON

★ What is String ?

A string is a sequence of characters enclosed in single quotes (' ') or double quotes (" ").

Example : `s = "Hello, Python!"`

★ String Indexing

Each character in a string has an index starting from 0.

`s = "Python"`

	P	y	t	h	o	n
Index :	0	1	2	3	4	5

★ String Slicing

Syntax : `s[start : end : step]`

- start → starting index (inclusive)
- end → ending index (exclusive)
- step → step size (default is 1)

Examples :

```
s = "Python"
s[0:3]      # 'Pyt'
s[2:5]      # 'tho'
s[:4]       # 'Pyth'
s[3:]       # 'hon'
s[::2]      # 'Pto'
s[:--1]     # 'nohtyP' (reverse string)
```

★ Immutable Strings

Strings are immutable, meaning once a string is created, it cannot be changed.
Any modification creates a new string.

```
s = "Python"
s[0] = 'J'        # Error
s = "J" + s[1:]   # 'Jython' (new string)
```

★ Common String Methods

Method	Description	Example
upper()	Converts to uppercase	s.upper() # 'PYTHON'
lower()	Converts to lowercase	s.lower() # 'python'
strip()	Removes spaces from both sides	" hi ".strip() # 'hi'
replace(old, new)	Replaces old with new	s.replace('t','T') # 'PyThon'
split(sep)	Splits string into list	"a,b,c".split(',') # ['a','b','c']
find(sub)	Returns first index of substring, -1 if not found	s.find('th') # 2
count(sub)	Counts occurrences of substring	s.count('o') # 1
startswith(sub)	Checks if string starts with substring	s.startswith('Py') # True
endswith(sub)	Checks if string ends with substring	s.endswith('on') # True
join(iterable)	Joins elements using string as separator	'-'.join(['a','b','c']) # 'a-b-c'

★ String Formatting

We can insert variables inside strings.

- f-string (Python 3.6+)

```
name = "Guest"
age = 25
msg = f"Hello {name}, you are {age} years old."
print(msg)    # Hello Guest, you are 25 years old.
```

Other ways

- Using format()

```
msg = "Hello {}, you are {} years old.".format(name, age)
```

- Using % operator

```
msg = "Hello %s, you are %d years old." % (name, age)
```

Note :

✓ Strings are case-sensitive.

✓ Use meaningful variable names for better code readability.

Python Full Handbook

PART 2

3. Lists in Python

★ What is List ?

A list is an ordered collection of items. It can store multiple values of different data types.

Note :
Lists are mutable (can be changed).

Creating a List

```
empty_list = [ ]              # empty list
nums = [1, 2, 3, 4, 5]        # numbers
mixed = [10, "hello", 3.14, True]  # mixed types
```

★ Indexing in List

```
nums = [10, 20, 30, 40, 50]
```

Positive Index :

0	1	2	3	4

Negative Index :

-5	-4	-3	-2	-1

```
nums[0]     # 10 (first element)
nums[2]     # 30 (third element)
nums[-1]    # 50 (last element)
nums[-3]    # 30 (third last element)
```

★ Slicing in List

```
nums = [10, 20, 30, 40, 50, 60]
```

- `nums[1:4]` # [20, 30, 40]
- `nums[:3]` # [10, 20, 30]
- `nums[3:]` # [40, 50, 60]
- `nums[::2]` # [10, 30, 50]
- `nums[::-1]` # [60, 50, 40, 30, 20, 10]

★ Common List Methods

Method	Description	Example	Output
append(x)	Adds an element at the end	lst.append(6)	[1, 2, 3, 4, 5, 6]
extend(iterable)	Adds multiple elements	lst.extend([6,7])	[1, 2, 3, 4, 5, 6, 7]
insert(i, x)	Inserts element x at index i	lst.insert(2, 99)	[1, 2, 99, 3, 4, 5]
remove(x)	Removes first occurrence of x	lst.remove(3)	[1, 2, 4, 5]
pop([i])	Removes and returns element at index i	lst.pop() lst.pop(1)	5 2
clear()	Removes all elements	lst.clear()	[ ]
index(x)	Returns first index of x	lst.index(4)	3
count(x)	Returns count of x in list	lst.count(2)	1
sort()	Sorts the list in ascending order	lst.sort()	[1, 2, 3, 4, 5]
reverse()	Reverses the list	lst.reverse()	[5, 4, 3, 2, 1]
len(lst)	Returns length of the list	len(lst)	5

List Comprehension (Quick Intro)

List comprehension provides a shorter syntax to create lists.

Example :

```
squares = [x*x for x in range(1, 6)]
print(squares)   # [1, 4, 9, 16, 25]
```

★

Key Points :

- ✓ Lists are ordered.
- ✓ Allow duplicates.
- ✓ Can store different data types.
- ✓ Indexing starts from 0.
- ✓ Negative index starts from -1.
- ✓ Lists are mutable.

Example :

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
fruits.insert(1, "mango")
fruits.remove("banana")
print(fruits)
# Output : ['apple', 'mango', 'cherry', 'date']
```

PYTHON FULL HANDBOOK

PART 2

4. TUPLE, SET & DICTIONARY IN PYTHON

★ TUPLE

- Tuple is an ordered collection of items.
- It is immutable (cannot be changed).
- Written using ( ) (parentheses).

Examples :

```
t = (10, 20, "hello", 3.14)
t[0]    # 10
t[-1]   # 3.14
```

★ SET

- Set is an unordered collection of unique items.
- It does not allow duplicates.
- Written using {} (curly braces).

Examples :

```
s = {1, 2, 3, 4, 5, 5, 3}
s              # {1, 2, 3, 4, 5}
s.add(6)       # {1, 2, 3, 4, 5, 6}
s.remove(2)    # {1, 3, 4, 5, 6}
```

★ DICTIONARY

- Dictionary stores data in key-value pairs.
- Keys must be unique and immutable.
- Written using { } with key : value.

Examples :

```
d = {"name": "Guest", "age": 25}
d["name"]    # 'Guest'
d["age"]     # 25
d["city"] = "Delhi"    # add new key
```

★ COMPARISON – List vs Tuple vs Set vs Dictionary

Feature	List	Tuple	Set	Dictionary
Order	Ordered	Ordered	Unordered	Unordered (by key)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not Allowed	Keys: Not Allowed Values: Allowed
Syntax	[ ]	( )	{ }	{ key : value }
Example	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{'a': 1, 'b': 2}
Access	By index	By index	No index	By key

Common Set Methods

add(x)	→ Adds an element
remove(x)	→ Removes an element
discard(x)	→ Removes if present (no error)
union(s)	→ Returns union of sets
intersection(s)	→ Returns common elements
difference(s)	→ Returns elements in set not in another set

Common Dictionary Methods

keys()	→ Returns all keys
values()	→ Returns all values
items()	→ Returns key-value pairs
update(d)	→ Updates dictionary with another dict
get(key)	→ Returns value for key (None if not found)
pop(key)	→ Removes key and returns its value

Examples

```
# Tuple example
t = (10, 20, 30)
print(t[1])       #  20
# Set example
s = {1, 2, 3, 4}
s.add(5)
print(s)          # {1, 2, 3, 4, 5}
# Dictionary example
d = {"name": "Guest", "age": 25}
d["age"] = 26
print(d)          # {'name': 'Guest', 'age': 26}
```

Note :

- ✓ Use List when order is important and duplicates are allowed.
- ✓ Use Tuple when data should not change.
- ✓ Use Set when you need unique values and fast operations.
- ✓ Use Dictionary when you need to store data in key-value pairs.

Python Full Handbook

PART 2

5. Loops in Python

★ For Loop

A for loop is used to iterate over a sequence (list, tuple, string, set, range etc.)

Syntax :

```
for variable in sequence:
    # block of code
```

Example :

```
for i in range(1, 6):
    print(i)    # Output : 1 2 3 4 5
```

range() Function

range(start, stop, step)

- start → starting value (default 0)
- stop → stopping value (exclusive)
- step → step size (default 1)

★ While Loop

A while loop is used to execute a block of code as long as condition is True.

Syntax :

```
while condition:
    # block of code
```

Example :

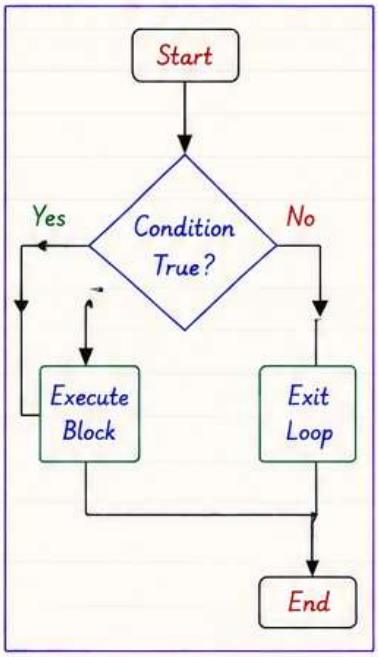
```
i = 1
while i <= 5:
    print(i)
    i = i + 1    # 1 2 3 4 5
```

★ Nested Loop

A loop inside another loop.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

Loop Flowchart



★ Control Statements in Loops

Statement	Description	Example	Output

break	Terminates the loop completely.	for i in range(1, 6):
		  if i == 3:
		  break
		  print(i)
			1 2

continue	Skips the current iteration and continues with next.	for i in range(1, 6):
		  if i == 3:
		  continue
		  print(i)
			1 2 4 5

pass	Does nothing. Used as a placeholder.	for i in range(1, 4): pass # No output	No output

★ Practice Examples

1. Print first 5 natural numbers

```
for i in range(1, 6):
    print(i)          # Output : 1 2 3 4 5
```

2. Print even numbers between 1 to 10

```
for i in range(2, 11, 2):
    print(i)          # Output : 2 4 6 8 10
```

3. Calculate sum of first n natural numbers

```
n = 5
sum = 0
for i in range(1, n+1):
    sum = sum + i
print(sum)            # Output : 15
```

Key Points :

- Use for loop when number of iterations is known.
- Use while loop when number of iterations is not known.
- break stops the loop.
- continue skips the current iteration.
- pass is used when a statement is syntactically required but we don't need any action.

Note :
Loops help us automate repetitive tasks and make our code efficient and clean.

Python Full Handbook

PART 2

6. Exception Handling & File Handling

★ Exception Handling in Python

Exception handling is used to handle runtime errors and keep the program from crashing.

Syntax :

```
try:
    # block of code
except ExceptionType:
    # block of code
else:
    # block of code
finally:
    # block of code
```

Example :

```
a = 10
b = 0
try:
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Division Successful")
finally:
    print("Execution Finished")
```

Key Points :

- ✓ try block contains code that might raise an error.
- ✓ except block handles the specific error.
- ✓ else block runs if no error occurs.
- ✓ finally block runs always, whether error occurs or not.
- ✓ Use specific exceptions instead of general Exception.

★ File Handling in Python

File handling allows us to read from or write to files.

Syntax :

```
f = open('filename', 'mode')   # open file
data = f.read()                # read data
f.write("text")                # write data
f.close()                      # close file
```

File Modes

Mode	Meaning
'r'	Read mode. File must exist.
'w'	Write mode. Creates new file or overwrites existing file.
'a'	Append mode. Adds data at the end of file.
'r+'	Read and write mode.
'w+'	Write and read mode.
'a+'	Append and read mode.
'x'	Create mode. Fails if file already exists.

Reading from a File

```
f = open('sample.txt', 'r')
data = f.read()      # read entire file
print(data)
f.close()
```

Writing to a File

```
f = open('sample.txt', 'w')
f.write("Hello Python!")
f.close()
```

Appending to a File

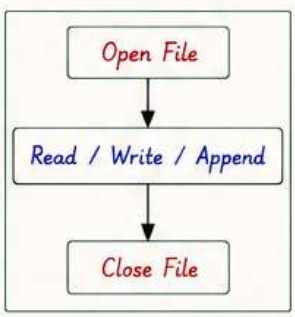
```
f = open('sample.txt', 'a')
f.write("\nThis is new line.")
f.close()
```

Using 'with' Statement (Best Practice)

```
with open('sample.txt', 'r') as f:
    data = f.read()
    print(data)

# File is closed automatically
```

File Handling Flow



Note :

- Always close file after use.
- Use 'with' statement to automatically close file.
- Handle exceptions while working with files.
- Choose correct mode based on operation.

Python Full Handbook

PART 2

7. List Comprehension, Lambda & Built-in Functions

★ List Comprehension

List comprehension provides a shorter and cleaner way to create lists.

Syntax :

```
[expression for item in iterable if condition]
```

Examples :

- `squares = [x*x for x in range(6)]`
  `# [0, 1, 4, 9, 16, 25]`
- `even = [x for x in range(1,11) if x % 2 ==0]`
  `# [2, 4, 6, 8, 10]`
- `words = [w for w in 'hello python'.split()]`
  `# ['hello', 'python']`
- `nums = [x for x in range(1,11) if x > 5]`
  `# [6, 7, 8, 9, 10]`

Note :

- ✓ List comprehension is faster and more readable.
- ✓ Use it when creating or transforming lists.

★ Lambda Function

Lambda is an anonymous (unnamed) function.

Syntax :

```
lambda arguments : expression
```

Examples :

- `square = lambda x : x*x`
- `add = lambda a, b : a + b`
- `max_num = lambda a, b : a if a > b else b`

Built-in Functional Tools

Function	Description	Example	Output
map()	Applies a function to all items	list(map(str, [1,2,3]))	['1', '2', '3']
filter()	Filters items using a function	list(filter(lambda x: x%2==0, [1,2,3,4,5,6]))	[2, 4, 6]
zip()	Combines multiple iterables	list(zip(['a', 'b', 'c'], [1,2,3]))	[('a', 1), ('b', 2), ('c', 3)]
enumerate()	Returns index and item together	list(enumerate(['a', 'b', 'c']))	[(0, 'a'), (1, 'b'), (2, 'c')]
sorted()	Returns a sorted list	sorted([3,1,4,2])	[1, 2, 3, 4]
sum()	Returns sum of all items	sum([1,2,3,4])	10

8. Modules, Packages & Virtual Environment

★ Modules

A module is a file containing Python code (functions, variables, classes).

Importing modules

```
import math
print(math.sqrt(16))      # 4.0
```

Import specific items

```
from math import pi
print(pi)      # 3.141592653589793
```

Creating your own module

```
# mymodule.py
def greet(name):
    return "Hello " + name
```

Using your module

```
import mymodule
print(mymodule.greet("Friend"))
# Hello Friend
```

★ Packages

A package is a collection of modules inside a directory.

Import from a package

```
from datetime import date
today = date.today()
print(today)      # 2024-05-20
```

Note :

Every package must contain `__init__.py` file (even if it's empty).

pip (Python Package Installer)

- Helps us install and manage packages.
- Check version : `pip --version`
- Upgrade pip : `pip install --upgrade pip`

Installing Packages

```
pip install requests
pip install pandas
pip install numpy
```

Virtual Environment

Virtual environment helps us create an isolated Python environment for a project.

Create Virtual Environment

```
python -m venv myenv
```

Activate Virtual Environment

```
# Windows
myenv\Scripts\activate
# Mac / Linux
source myenv/bin/activate
```

Project Environment Flow

Project Folder → Virtual Environment → Installed Packages → Your Code

Benefits of Virtual Environment

- Keeps project dependencies separate.
- Avoids version conflicts.
- Makes projects portable and clean.

Python Full Handbook

PART 2

8. Modules, Packages & Virtual Environment (Contd.)

★ Importing in Different Ways

- `import module`
  ```
  import math
  math.sqrt(25)      # 5.0
  ```
- `from module import name`
  ```
  from math import sqrt
  sqrt(16)           # 4.0
  ```
- `from module import name as alias`
  ```
  from math import sqrt as root
  root(9)            # 3.0
  ```
- `import module as alias`
  ```
  import numpy as np
  arr = np.array([1, 2, 3])
  ```
- `from module import *` (Not Recommended)
  ```
  from math import *
  print(pi)          # 3.14159...
  ```

★ Common Built-in Modules

Module	Use
math	Mathematical functions
random	Random numbers
datetime	Date and time
os	Operating system interactions
sys	System specific parameters
json	Work with JSON data
re	Regular expressions
collections	Specialized container datatypes

★ Creating Your Own Package

Project Structure

```
mypackage/
    → __init__.py
    → mymodule.py
        └→ def add(a, b):
               return a + b
    → main.py
```

Using the Package

```
from mypackage.mymodule import add
result = add(5, 7)
print(result)   # 12
```

Common Special Variables in Modules

Variable	Meaning
`__name__`	Name of the current module
`__file__`	Path of the current file
`__doc__`	Documentation string of module
`__package__`	Name of the package
`__all__`	List of public names in module

Python Search Path

Python looks for modules in the following order:

→ Current directory
→ PYTHONPATH environment variable
→ Standard library directories
→ Site-packages directory

Useful Commands

- `dir(module)` → list of attributes of a module
- `help(module)` → help on module
- `print(sys.path)` → show search path
- `pip list` → list installed packages

Virtual Environment – Complete Guide

1. Create
`python -m venv env`

→

2. Activate
Windows:
`env\Scripts\activate`
Mac / Linux:
`source env/bin/activate`

→

3. Install Packages
`pip install pandas`
`pip install requests`

→

4. Work
Use Python & installed packages

→

5. Deactivate
`deactivate`

Why Use Virtual Environment?

- Isolates project dependencies.
- Avoids version conflicts.
- Keeps global Python clean.
- Makes projects portable.

Deactivate Commands

- Windows : deactivate
- Mac / Linux: deactivate

Best Practices

- Always create a virtual environment for every project.
- Keep requirements in requirements.txt using pip freeze > requirements.txt
- Activate environment before working on the project.
- Do not commit virtual environment folder to Git.

Quick Summary – Part 2

- ✓ Functions help in code reusability.
- ✓ Strings are immutable and have many useful methods.
- ✓ Lists are mutable and dynamic.
- ✓ Tuples are immutable.
- ✓ Sets store unique values.
- ✓ Dictionaries store key-value pairs.
- ✓ Loops help us iterate over sequences.
- ✓ Exception handling makes code robust.
- ✓ Files allow permanent data storage.
- ✓ List comprehension simplifies list creation.
- ✓ Lambda functions are small anonymous functions.
- ✓ Built-in functions save time.
- ✓ Modules help organize code.
- ✓ Packages group related modules.
- ✓ Virtual environments manage dependencies.

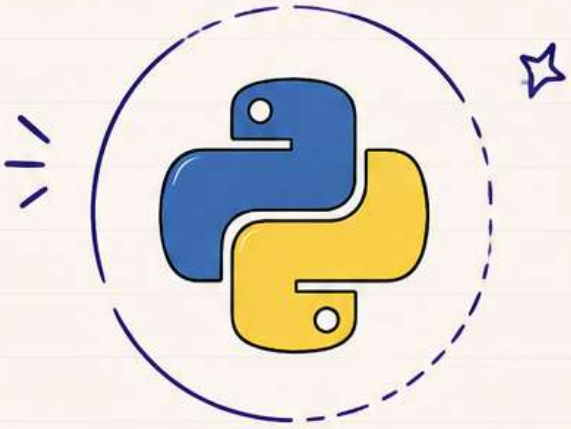
KEEP PRACTICING, KEEP CODING, KEEP GROWING! ★

COMMENT

PYTHON

TO GET

COMPLETE PDF



♥ SAVE | SHARE | FOLLOW ♥

aman_views